MMdedup: A Parallel and Pipelined Multimodal Data Deduplication Framework for MLLM Training

Zhuoyang Wu
Sun Yat-sen University
wuzhy236@mail2.sysu.edu.cn

Qingli Zeng
University of South Carolina Upstate
qingli@uscupstate.edu

Yinjin Fu
Sun Yat-sen University
fuyj27@mail.sysu.edu.cn

Nong Xiao
Sun Yat-sen University
xiaon6@mail.sysu.edu.cn

ABSTRACT

Deduplication is an essential data cleaning technique to enhance data quality for multimodal large language model (MLLM) training by identifying and removing redundant or duplicate data. Existing data deduplication methods usually assume that data is pre-sorted or focus only on a single modality, which creates a critical gap when handling raw, heterogeneous datasets. In this paper, we propose MMdedup, a highly efficient multimodal data deduplication framework to improve the efficiency of MLLM training and data storage. It designs a parallel and pipelined classification-and-clean architecture, which first employs a lightweight heuristic sorter to automatically execute robust data classification. Subsequently, MMdedup performs modality-specific deep cleaning techniques to eliminate both exact-duplicate and near-duplicate data for each modality. We evaluated MMdedup on TB-scale datasets via prototype implementation. Experimental results demonstrate that our sorter achieves 100% classification accuracy on constructed adversarial inputs, while deduplication modules reduce storage by 46-90% depending on modality. Compared to state-of-the-art methods, our image deduplication achieves 3.6× higher throughput while improving downstream model accuracy, our audio deduplication can achieve the highest downstream accuracy of 92.01%, and our text deduplication matches MinHash quality with 14% faster processing. Ablation studies confirm that our near-duplicate detection framework contributes over 20 percentage points of additional deduplication beyond exact matching, validating our core design.

PVLDB Reference Format:
Zhuoyang Wu, Qingli Zeng, Yinjin Fu, and Nong Xiao. MMdedup: A Parallel and Pipelined Multimodal Data Deduplication Framework for MLLM Training. PVLDB, 14(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/xiaoyu-ops/VLDB26.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

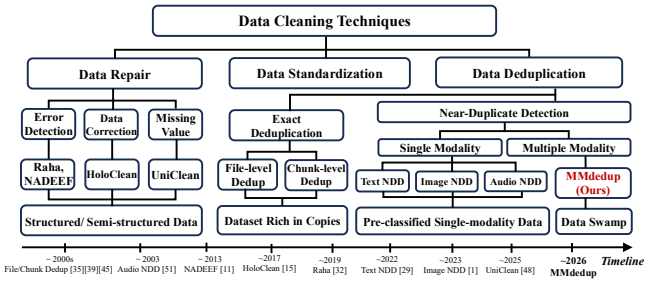


Figure 1: Data cleaning techniques taxonomy.

1 INTRODUCTION

Data quality and diversity are crucial for large language model (LLM) training, as they directly impact model performance, accuracy, and ethical considerations in AI development. As the LLMs grow increasingly sophisticated, capable of generating human-like text and tackling complex tasks, the need for meticulously curated datasets becomes paramount [52, 55]. Multimodal large language model (MLLM) [54, 57] can perform tasks that were previously challenging or impossible for single-modality models by processing and integrating multiple types of data or sensory inputs, such as text, images and audio. Web-crawled data has become the primary source of training data for MLLMs due to its large-scale and diversity [20, 42]. However, these massive amounts of raw web data rarely conform to clean, structured formats that can be immediately used by AI models. Instead, they often manifest as a chaotic 'digital swamp' which is an unstructured, heterogeneous data collection, posing two serious data engineering challenges. Firstly, these datasets exhibit inherent heterogeneity, intermixing diverse modalities, such as images, audio, text, ambiguous binary files, and metadata links within a single corpus, etc. Handling such heterogeneity under resource constraints is a critical infrastructure challenge. Just as CompactST [22] introduces specialized normalizers to tackle domain heterogeneity in data-scarce settings, we introduce a specialized 'Classification' mechanism to disentangle format heterogeneity at the ingestion stage. Secondly, these data collections suffer from pervasive redundancy, encompassing both exact duplicates and, more subtly, near-duplicate content arising from templates, paraphrasing, or cross-site aggregation. Recent work [29] demonstrated high redundancy even in seemingly curated datasets. Such data redundancy not only increases computational costs, but also damages the AI model's generalization ability by cultivating undesirable memorization.

As illustrated in Figure 1, existing training data cleaning methods can be broadly categorized into three independent lines of work. Despite the widely recognized importance of data quality, existing data preprocessing and cleaning methods often struggle to cope with the complexity of the digital swamp [10]. State-of-the-art data cleaning frameworks [11, 15, 32, 36, 44, 48] mainly focus on detecting and repairing dirty data within structured or semi-structured datasets, implicitly assuming pre-classified unimodal inputs. Traditional data deduplication techniques [18, 53] target byte-identical redundancy removal at the file or chunk level, using cryptographic hashes such as MD5 [45] and SHA-256 for file-level matching, or content-defined chunking via Rabin fingerprinting [35, 39] for finer-grained deduplication.

Moreover, although sophisticated techniques for near-duplicate detection (NDD) within specific modalities, such as the suffix-array and MinHash approaches for text [29], semantic embedding methods for images [1], and acoustic fingerprinting techniques originally developed for audio retrieval [51], have made significant progress, they each operate on a single pre-classified modality and lack an integrated, automated front end that can handle the initial chaos of raw, multimodal web harvests.

In this paper, we introduce MMdedup, a parallel and pipelined multimodal data deduplication framework for data cleaning in MLLM training. It automatically transforms raw and chaotic web data into high-quality, non-redundant corpora optimized for downstream machine learning applications by leveraging the "classification-before-clean" philosophy. The MMdedup framework consists of two different stages: *automatic classification* and *modality-aware data deduplication*. Firstly, it designs a lightweight but powerful heuristic sorter to reliably classify raw input files into modality specific subsets by utilizing a range of content driven checks, including magic sniffing, printability analysis, and JSON semantic parsing. This observation-driven design aligns with recent system research like LogLite [49], which proves that simple heuristic features (e.g., log length) are often sufficient to achieve wire-speed processing in streaming tasks, avoiding the latency overhead of heavy models. Secondly, MMdedup utilizes a modality-aware data deduplication technique to carefully select the modality specific algorithm for each classification subset, such as the semantic understanding of CLIP (Contrastive Language-Image Pretraining) embedding for image deduplication [40], our newly proposed robust and scalable acoustic fingerprinting for audio deduplication, and the multi-granularity N-gram method to capture syntactic similarity for text deduplication [2]. The main novelty lies not only in the design of these individual components, but also in their collaborative integration to validate as an end-to-end solution for the MLLM training data cleaning.

To the best of our knowledge, MMdedup is the first research work on multimodal data deduplication for MLLM training. Our main contributions are as follows:

- We propose MMdedup, a parallel and pipelined multimodal data deduplication framework that transforms raw, heterogeneous web data into clean training corpora by eliminating both exact and near-duplicate content across modalities.
- We design a lightweight heuristic sorter that achieves 100% classification accuracy on adversarial inputs while maintaining high throughput of 4.25K files/second.
- We introduce an acoustic fingerprinting based audio deduplication method and a modality-aware data deduplication technique by integrating the advanced modality-specific deep cleaning techniques to remove near-duplicate after exact deduplication.
- We implement a prototype of MMdedup and conduct comprehensive experiments on datasets totaling nearly 1TB. Our results demonstrate significant data reduction while maintaining or improving downstream model accuracy.

2 RELATED WORK

The selection of high-quality training data has emerged as a critical bottleneck for large-scale machine learning, particularly for multimodal large language models (MLLMs). As training corpora scale to billions of samples sourced from heterogeneous web crawls, the challenges of data redundancy, quality control, and efficient processing become increasingly pronounced. We organized related work into five areas: data cleaning, text deduplication, image deduplication, audio deduplication, and file type identification.

2.1 Data Cleaning

Data cleaning is a broad field encompassing tasks such as error detection and correction, missing value imputation, and data deduplication. Existing data cleaning systems, such as HoloClean [44] and UniClean [15], have revolutionized structured data repair through probabilistic inference and unified operator optimization. However, these state-of-the-art systems generally operate on datasets that already possess some structure (e.g., relational tables) or conform to a known schema. Their primary focus is often on value correctness and exact duplicates, with less emphasis on near-duplicate removal across collections of unstructured files. What is more, they lack an automated data classification mechanism to handle raw input data, and often assume that the input datasets are already modality-specific and somewhat organized, differing significantly from the initial state of real-world web data. Hence, these works are inherently ill-suited for the chaotic 'digital swamp' of raw web dumps, which requires a robust and modality-aware classification mechanism that MMdedup provides as a prerequisite step.

Data deduplication is a foundational data cleaning technique focused on removing redundant or duplicate data to improve data quality, save storage space, and optimize data transfers. Traditional data deduplication only focuses on exact duplicate removal to improve communication or storage efficiency for various application datasets [18, 53]. As demonstrated by [29], we can further improve data quality and processing efficiency for LLM training by identifying and eliminating near-duplicate content beyond exact deduplication. Hence, our MMdedup framework integrates optimized near-duplicate detection (NDD) techniques tailored for different data modalities.

2.2 Text Deduplication

The impact of data deduplication on language model training has been extensively studied in recent years. Lee et al. [29] provided foundational empirical evidence that deduplicating training data significantly improves language model quality. Using exact substring matching via suffix arrays and approximate document-level matching via MinHash, they demonstrated that deduplicated models emit

Table 1: Comparison of MMdedup with representative deduplication approaches.

Method	Input Handling			Deduplication			Pipeline	
	Mixed	Auto	Raw	Text	Image	Audio	E2E	Config.
Text Deduplication								
MinHash-LSH [2]	✗	–	✗	✓	✗	✗	✗	✗
D4 [50]	✗	–	✗	✓	✗	✗	✗	✓
LSHBloom [27]	✗	–	✗	✓	✗	✗	✗	✗
Image Deduplication								
pHash [56]	✗	–	✗	✗	✓	✗	✗	✗
SemDeDup [1]	✗	–	✗	✗	✓	✗	✗	✓
Audio Deduplication								
Shazam [51]	✗	–	✗	✗	✗	✓	✗	✗
Neural Audio FP [6]	✗	–	✗	✗	✗	✓	✗	✗
File Type Detection								
libmagic [12]	✓	✓	✓	–	–	–	✗	✗
Magika [17]	✓	✓	✓	–	–	–	✗	✗
MMDEDUP (Our Paper)	✓	✓	✓	✓	✓	✓	✓	✓

✓ supported; ✗ not supported; – not applicable.

memorized content 10× less frequently while achieving equivalent or better perplexity. This work established deduplication as a standard preprocessing step for LLM training pipelines.

MinHash combined with Locality-Sensitive Hashing (LSH) [2, 3] has become the de facto standard for large-scale near-duplicate detection. By approximating Jaccard similarity between document shingle sets, MinHash-LSH enables sub-quadratic complexity for candidate pair identification. The technique has been widely adopted in major training pipelines, including GPT-3 [4], which applied fuzzy deduplication to Common Crawl using Spark's MinHashLSH implementation. Recent work by Khan et al. [27] introduced LSHBloom, which replaces the traditional LSH index with Bloom filters to achieve a 12× speedup and an 18× reduction in memory usage. These improvements are critical for datasets exceeding 10TB.

Beyond syntactic similarity, semantic deduplication identifies samples with similar meaning but different surface forms. Tirumala et al. [50] proposed D4, which combines embedding-based clustering with diversity-aware sampling. By using pre-trained model embeddings to identify semantically redundant documents, D4 achieved 20% training efficiency gains and up to 2% accuracy improvements on downstream NLP tasks at the 6.7B parameter scale. The C4 corpus [42], created for T5 training, pioneered large-scale web text deduplication by applying heuristic filtering and three-sentence span deduplication to Common Crawl, producing a 750GB clean corpus from 20TB of raw data.

2.3 Image Deduplication

Image deduplication presents distinct challenges compared to text, as visual similarity operates at multiple levels ranging from pixel-level identity to semantic equivalence. Traditional approaches employ perceptual hashing algorithms such as pHash and dHash [56], which generate compact fingerprints robust to minor transformations like resizing, compression, and color adjustment. These methods offer $O(n)$ complexity for fingerprint generation and $O(1)$ comparison via Hamming distance, making them suitable for large-scale deployment. However, perceptual hashes struggle with semantic similarity. For example, two images of different dogs can produce very different hashes even though they are semantically equivalent.

Recent advances leverage deep learning embeddings for semantic deduplication. Abbas et al. [1] introduced SemDeDup, which uses pre-trained CLIP embeddings to identify image pairs that are semantically similar but not visually identical. On LAION-440M, SemDeDup removed 50% of data with minimal performance loss, effectively halving training time while improving out-of-distribution generalization. The key insight is that foundation model embeddings capture semantic similarity that traditional hashing misses, enabling more aggressive deduplication without quality degradation.

For web-scale datasets, exact pairwise comparison is intractable. SemDeDup addresses this through k-means clustering followed by intra-cluster deduplication, reducing complexity from $O(n^2)$ to $O(n^2/k)$. The DataComp benchmark [19] further demonstrated that combining CLIP-score filtering with embedding-based deduplication produces superior training datasets; their DataComp-1B subset trains a ViT-L/14 to 79.2% ImageNet zero-shot accuracy, outperforming models trained on the full LAION-2B.

2.4 Audio Deduplication

Audio deduplication relies on acoustic fingerprinting techniques that extract compact representations robust to noise and compression. The foundational work by Wang [51] introduced spectrogram peak-based fingerprinting, where local maxima in the time-frequency domain are paired and hashed to create distinctive signatures. This approach, commercialized as Shazam, remains influential due to its robustness against background noise and audio degradation. Subsequent systems such as Chromaprint [28] adopted similar constellation-based methods optimized for near-duplicate detection

in music libraries. Recent advances leverage deep learning for more robust fingerprinting. Chang et al. [6] proposed a neural audio fingerprinting system based on contrastive learning, achieving 10× storage reduction while maintaining retrieval accuracy under various distortions including noise, reverberation, and codec compression.

Despite these advances, audio deduplication has received less attention than text and image modalities in the context of ML training data curation. Existing multimodal dataset efforts such as LAION-5B and DataComp focus primarily on image-text pairs, leaving audio data, which is increasingly important for speech and multimodal models, largely unaddressed.

2.5 File Type Identification

Accurate file type classification is essential for routing heterogeneous data to appropriate processing pipelines. The traditional approach relies on magic bytes, which are characteristic byte patterns at known file offsets. McDaniel and Heydari [33] systematically evaluated content-based file type detection algorithms, including byte frequency analysis and header/trailer matching, reporting accuracy ranging from 23% to 96% depending on the algorithm and file type. Li et al. [31] introduced file fingerprints using n-gram analysis, achieving improved robustness over simple signature matching. Modern implementations such as libmagic maintain over 5,000 signatures, achieving 95–100% accuracy on well-formed files with standard headers [21].

Despite their efficiency, signature-based methods struggle with several scenarios common in web-crawled data. Gopal and Yang [21] demonstrated that commercial off-the-shelf solutions fail catastrophically on damaged files, with some unable to identify corrupted files at all. Key limitations include corrupted headers from incomplete downloads, missing or misleading file extensions, text-based formats (JSON, CSV, plain text) that lack distinctive signatures, and polyglot files valid under multiple formats.

To address these limitations, researchers have explored neural approaches to content-based file type detection. Raff et al. [41] introduced MalConv, a convolutional neural network that processes raw byte sequences of up to two million time steps, demonstrating that deep learning can learn discriminative patterns directly from binary content without hand-crafted features. Building on this direction, Fratantonio et al. [17] presented Magika, a lightweight deep learning model achieving 99% average F1 score across 113 content types while requiring only 1MB of memory and 5ms inference time per file. Magika has been deployed at scale for Gmail attachment scanning and VirusTotal malware analysis, processing hundreds of billions of files weekly. While neural approaches offer superior accuracy, they incur computational overhead that creates tension between accuracy and throughput in high-volume pipelines.

2.6 Summary and Positioning

Table 1 summarizes the key differences between MMdedup and representative prior works across three dimensions: input handling, deduplication capabilities, and pipeline integration. As shown in the table, existing deduplication methods operate on pre-sorted, single-modality inputs, while file type detection tools function as standalone utilities without deduplication capabilities. MMdedup bridges these gaps by providing an end-to-end solution that (1) accepts raw heterogeneous inputs without manual preprocessing, (2) integrates lightweight file classification with modality-specific deduplication for text, image, and audio, and (3) offers configurable thresholds to balance deduplication aggressiveness against downstream task performance. Rather than proposing novel algorithms for individual components, our contribution lies in their systematic integration and empirical validation as a cohesive framework for preparing multimodal training data at scale.

3 SYSTEM DESIGN

We present MMdedup, an end-to-end framework that transforms raw, heterogeneous web-crawled data into high-quality, deduplicated, modality-specific corpora for machine learning. As depicted in Figure 2, MMdedup follows a Classification-and-Clean architecture comprising two phases. In Phase 1, a lightweight content-aware classifier, the Sorter, ingests the unstructured corpus and routes each file into one of three modality streams (text, image, or audio) through content inspection rather than unreliable file extensions. In Phase 2, each stream passes through a two-pass deduplication pipeline: a universal exact deduplication pass removes bit-for-bit identical copies, after which a near-deduplication pass employs modality-specific algorithms to detect semantically or structurally equivalent files that differ at the byte level.

3.1 Design Rationale

MMdedup is guided by a pragmatic design philosophy that trades extreme semantic granularity for the high throughput demanded by petabyte-scale ingestion. Deep cross-modal alignment, exemplified by SyntheClean's [9] adaptive latent-space fusion, offers appealing semantic precision, yet its iterative cost is prohibitive for the initial processing of a raw web crawl. Our lightweight-first stance draws support from high-throughput designs such as LogLite [49], which show that deterministic statistical features suffice for robust data routing without the overhead of deep learning classifiers at the ingestion stage.

From this principle emerges a hybrid-depth strategy that matches algorithmic sophistication to the dominant redundancy type in each modality. Image duplicates are frequently semantic, meaning different photographs of the same object under varying conditions, and therefore warrant the cost of a deep embedding model (CLIP [40]). Text and audio redundancy, by contrast, is predominantly syntactic or acoustic, such as boilerplate paragraphs, mirrored articles, or identical recordings re-encoded at different bitrates. For these modalities, computationally lightweight methods, namely multi-granularity N-gram matching and spectral fingerprinting respectively, deliver high recall at a fraction of the cost. This stratified design positions MMdedup as a scalable first-pass filter that eliminates the bulk of low-level duplicates while preserving the option for deeper semantic cleaning in subsequent stages.

3.2 Phase 1: Content-Aware Classification

The Sorter implements a multi-layer, mixed-heuristic classification strategy that cascades from coarse binary signatures to fine-grained content-driven parsing. Its design targets the adversarial conditions endemic to web-scraped corpora: missing or incorrect file extensions, nested containers, encoding anomalies, and truncated files.

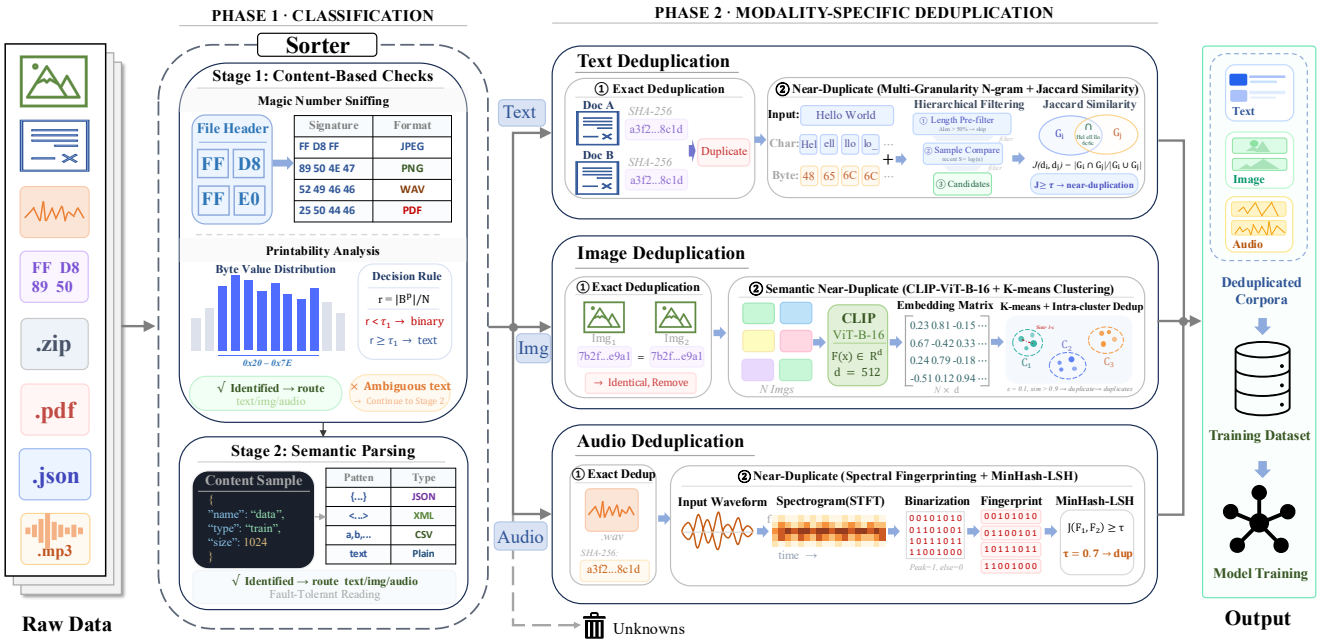


Figure 2: The MMdedup architecture.

Stage 1: content-based checks. Classification begins with magic number sniffing, which inspects the leading bytes of each file against a signature table of common binary formats. Files that match a known signature are immediately routed to the corresponding modality stream. Concurrently, a printability analysis is applied: the ratio $r = |B_p|/N$ of printable bytes B_p to the total byte count N is computed and compared against a threshold τ_1. Files with $r \geq \tau_1$ are classified as text candidates; those with $r < \tau_1$ are classified as binary. Files that remain ambiguous after this stage proceed to Stage 2.

Stage 2: semantic parsing. Ambiguous files undergo structural pattern matching, in which a content sample is inspected for recognizable formatting cues. As a final safeguard, fault-tolerant reading handles files with encoding anomalies or truncation by designating them as "unknown" rather than terminating with an error, ensuring pipeline continuity.

This hierarchical strategy yields significant advantages. It ensures robustness by reliably handling adverse conditions such as missing/incorrect file extensions, nested containers, or empty files. Crucially, it achieves an accuracy approaching that of machine learning models while maintaining the high throughput of a rule-based system. This design also preserves an extensible interface for the future integration of more complex detectors.

3.3 Phase 2: Modality-Specific Deduplication

Each classified stream first undergoes exact deduplication: a SHA-256 hash is computed for every file, and files sharing a hash are identified as bit-for-bit duplicates with only one copy retained. This lightweight pass eliminates a substantial fraction of trivially redundant data before the costlier near-deduplication stage. The following subsections describe the near-deduplication algorithm tailored to each modality; key notation is consolidated in Table 2.

Table 2: Notation summary

Scope	Symbol	Description
Text	$\mathcal{D}$	Input text corpus $\{d_1, \ldots, d_n\}$
	k	n-gram size (default: 3)
	$\mathcal{G}_i$	Fused n-gram set of text d_i
	S	Comparison window size
	τ	Jaccard threshold (default: 0.8)
Image	$\mathcal{I}$	Input image set $\{I_1, \ldots, I_n\}$
	d	CLIP embedding dimension (512)
	k	Number of k-means clusters (2,000)
	ϵ	Distance threshold (default: 0.1)
	T_{CLIP}	Per-image CLIP inference cost
Audio	$\mathcal{A}$	Input audio set $\{a_1, \ldots, a_n\}$
	$\mathbf{f}_i$	Binary fingerprint of sample i
	b, r	LSH bands (20) and rows per band (10)
	$h = b \cdot r$	MinHash signature length
	τ	Jaccard threshold (default: 0.7)

3.3.1 Text Near-Deduplication. The integrity of large-scale machine learning (ML) datasets is critically undermined by the widespread presence of near-duplicates. In line with our framework's design rationale of prioritizing computational efficiency for initial bulk cleaning, our text deduplication pipeline focuses on eliminating the most pervasive form of redundancy in web data: syntactic duplicates

Algorithm 1: Multi-Granularity N-gram Text Deduplication

Input: Text corpus $\mathcal{D} = \{d_1, \ldots, d_n\}$, N-gram size k, threshold τ, window size S
Output: Deduplicated corpus $\mathcal{D}'$

$\mathcal{D}' \leftarrow \emptyset$;
$\mathcal{K} \leftarrow \emptyset$; `// retained N-gram sets`
for $i \leftarrow 1$ **to** n **do**
  $t \leftarrow \textsc{Normalize}(d_i)$;
  $\mathcal{G}_i \leftarrow \textsc{CharNgrams}(t, k) \cup \textsc{WordNgrams}(t, k)$;
  $dup \leftarrow \textsc{False}$;
  foreach $\mathcal{G}_j \in \mathcal{K}[\max(0, |\mathcal{K}| - S) :]$ **do**
    if $\frac{|\mathcal{G}_i \cap \mathcal{G}_j|}{|\mathcal{G}_i \cup \mathcal{G}_j|} \geq \tau$ **then**
      $dup \leftarrow \textsc{True}$;
      break;
  if ***not*** dup **then**
    $\mathcal{D}' \leftarrow \mathcal{D}' \cup \{d_i\}$;
    $\mathcal{K} \leftarrow \mathcal{K} \cup \{\mathcal{G}_i\}$;
return $\mathcal{D}'$

(e.g., boilerplate, copy-pasted articles). While deep semantic models like SBERT [43] can capture paraphrasing, their computational overhead is prohibitive for a first-pass analysis on petabyte-scale corpora [29]. Our approach therefore employs a highly efficient multi-granularity N-gram method, which is specifically optimized to achieve maximum throughput while effectively capturing a broad range of syntactic and lexical variations.

Each text is preprocessed via case normalization, whitespace consolidation, removal of non-alphanumeric characters (preserving both CJK and Latin scripts), and whitespace trimming, ensuring that surface-level formatting artifacts do not bias downstream feature extraction.

We extract two complementary sets of trigrams (k=3). Character-level trigrams, produced by a sliding window over the normalized string, capture fine-grained orthographic and morphological patterns. Word-level trigrams, generated after tokenization, preserve phrase-level structure and word order. Their set union forms a composite feature set $\mathcal{G}_i$ per text d_i, offering richer discriminative power than either granularity in isolation.

Exhaustive pairwise comparison is infeasible at scale; we therefore employ a three-tier filtering mechanism. (i) A length pre-filter discards pairs whose text lengths differ by more than 50%. (ii) A sliding-window comparison limits each incoming text to the most recent S retained texts, with S set proportional to $\log n$, reducing amortized complexity from $O(n^2)$ to $O(n \log n)$. (iii) Surviving pairs are scored by the Jaccard coefficient over their fused N-gram sets:

$$\mathrm{sim}(d_i, d_j) = \frac{|\mathcal{G}_i \cap \mathcal{G}_j|}{|\mathcal{G}_i \cup \mathcal{G}_j|}, \tag{1}$$

and pairs exceeding a threshold $\tau = 0.8$ (selected via grid search) are classified as near-duplicates, with one copy retained at random.

To prevent data leakage, we enforce a retention priority of train → test → validation when deduplication spans dataset splits, guaranteeing partition independence for reliable model evaluation. The complete procedure is formalized in Algorithm 1, which integrates the above stages into a single streaming pass over the corpus. The time complexity of Algorithm 1 is $O(n \cdot S \cdot m)$, where m is the average N-gram set size. Setting $S \propto \log n$ yields $O(n\, m \log n)$, markedly better than the $O(n^2 m)$ cost of brute-force comparison. Space consumption is $O(n \cdot m)$, dominated by the retained N-gram sets. The approach is theoretically grounded in two properties: the Jaccard index normalizes for length variation, keeping comparisons meaningful across documents of disparate size; and the multi-granularity feature space provides resilience to synonym substitution, minor reordering, and superficial formatting changes.

3.3.2 Image Near-Deduplication. Pixel-level methods such as perceptual hashing [25] are brittle under geometric transforms, color shifts, and compression artifacts. To detect semantic near-duplicates, meaning images depicting the same subject under different conditions, we leverage deep visual embeddings. Recent work on cross-modal noise filtering [9] corroborates the effectiveness of latent-space geometry for data quality control; our pipeline builds on this insight by coupling CLIP-based embedding with a divide-and-conquer clustering strategy that renders pairwise comparison tractable at scale.

Each image is resized to 256×256, center-cropped to 224×224, and passed through the CLIP-ViT-B-16 encoder [40], yielding a 512-dimensional embedding that is L2-normalized to unit length for cosine similarity computation. GPU-accelerated batch inference enables processing at web scale.

We partition the embedding space into k=2,000 clusters via GPU-accelerated spherical k-means [14] implemented in Faiss [16], using the cosine distance metric, 100 iterations, and a fixed random seed for reproducibility. Each resulting cluster groups semantically related images, decomposing the global deduplication problem into a set of smaller, independent sub-problems.

Within each cluster C_j, we compute the upper-triangular pairwise cosine similarity matrix and, for each image I_i, record its maximum similarity score $M_i = \max_{l \in C_j, l \neq i} \mathbf{e}_i^\top \mathbf{e}_l$. An image is removed as a near-duplicate when $M_i > 1 - \epsilon$, with the distance threshold $\epsilon = 0.1$ (i.e., similarity > 0.9) chosen via grid search to balance precision and recall. Algorithm 2 formalizes this three-stage pipeline of embedding, clustering, and intra-cluster filtering.

The divide-and-conquer design of Algorithm 2 yields substantial savings. Feature extraction costs $O(n \cdot T_{\text{CLIP}})$; clustering, $O(n \cdot k \cdot n_{\text{iter}} \cdot d)$; and intra-cluster comparison, $\sum_j O(|C_j|^2) \approx O(n^2/k)$ for balanced clusters. Setting $k \approx \sqrt{n}$ yields an overall $O(n^{3/2})$ bound, a significant reduction from the $O(n^2)$ baseline. In practice, deduplicating 100K images on an NVIDIA RTX 3090 completes in 2-3 hours, representing a two-order-of-magnitude speedup over exhaustive comparison.

3.3.3 Audio Near-Deduplication. The predominant form of audio redundancy in web crawls consists of acoustically identical or near-identical recordings that differ only in noise floor, encoding format, or amplitude normalization. Semantic embedding models such as VGGish [24] and YAMNet [5] can relate recordings of similar events (e.g., two distinct dog barks), yet their computational cost is excessive for the high-throughput removal of direct acoustic duplicates. We instead cast the problem as binary vector similarity

Algorithm 2: Semantic Image Deduplication

Input: Image set $\mathcal{I} = \{I_1, \ldots, I_n\}$, cluster count k, distance thresholds $\mathcal{E} = \{\epsilon_1, \ldots, \epsilon_m\}$
Output: Deduplicated sets $\{\mathcal{I}'_{\epsilon_1}, \ldots, \mathcal{I}'_{\epsilon_m}\}$

for $i \leftarrow 1$ **to** n **do**
  $\mathbf{e}_i \leftarrow \text{L2Norm}(\text{CLIP}(I_i))$;
$\{C_1, \ldots, C_k\} \leftarrow \text{SphericalKMeans}(\{\mathbf{e}_i\}_{i=1}^n, k)$;
foreach $\epsilon \in \mathcal{E}$ **do**
  $\mathcal{I}'_\epsilon \leftarrow \emptyset$;
  for $j \leftarrow 1$ **to** k **do**
    foreach $i \in C_j$ **do**
      if $\max_{l \in C_j,\, l \neq i} \mathbf{e}_i^\top \mathbf{e}_l \leq 1 - \epsilon$ **then**
        $\mathcal{I}'_\epsilon \leftarrow \mathcal{I}'_\epsilon \cup \{I_i\}$;
return $\{\mathcal{I}'_{\epsilon_1}, \ldots, \mathcal{I}'_{\epsilon_m}\}$

matching and solve it through a three-phase pipeline of spectral fingerprinting, locality-sensitive hashing, and verification.

Each audio signal is transformed into the time-frequency domain via the short-time Fourier transform (STFT) [58] with a Hanning window (n_{fft}=2048, hop length=64), implemented with the Librosa library. The magnitude spectrogram is converted to a logarithmic scale to compress dynamic range and amplify weak spectral components. An adaptive binarization step then produces a noise-resilient representation: each time-frequency bin is set to 1 if its log-magnitude exceeds a locally computed Otsu threshold by 6 dB, and to 0 otherwise. To ensure dimensional consistency across recordings of varying length, the binary spectrogram is resampled to a canonical 32×128 grid and flattened into a 4,096-dimensional binary fingerprint vector $\mathbf{f}_i$.

We apply MinHash-based [2] locality-sensitive hashing (LSH) [13] to avoid exhaustive pairwise comparison. Each fingerprint is compressed into a MinHash signature of length $h = b \times r$ via random permutations. The signature matrix is partitioned into b bands of r rows; fingerprints that collide in any band form candidate near-duplicate pairs.

Every candidate pair is verified by computing the Jaccard similarity [30] on the original fingerprint vectors:

$$J(\mathbf{f}_i, \mathbf{f}_j) = \frac{|\mathbf{f}_i \cap \mathbf{f}_j|}{|\mathbf{f}_i \cup \mathbf{f}_j|}. \tag{2}$$

Pairs satisfying $J \geq \tau = 0.7$ are deemed near-duplicates; one member of each pair is removed at random. The full procedure is given in Algorithm 3. Algorithm 3 achieves $O(n \log n)$ time and $O(nh)$ space. The LSH framework provides a probabilistic detection guarantee: for a pair with true similarity s, the collision probability is $1 - (1 - s^r)^b$, and tuning b and r offers direct control over the recall–precision trade-off. The pipeline is resilient along three axes: (i) adaptive binarization with the 6 dB offset mitigates additive noise; (ii) log-scale transformation and thresholding absorb amplitude variations; and (iii) the overlapping STFT window design tolerates minor temporal shifts.

Algorithm 3: Audio Deduplication

Input: Audio set $\mathcal{A} = \{a_1, \ldots, a_n\}$, threshold τ=0.7, bands b=20, rows r=10
Output: Deduplicated set $\mathcal{A}'$

`// Phase 1: fingerprint generation`
for $i \leftarrow 1$ **to** n **do**
  $y \leftarrow \text{Load}(a_i)$;
  $D \leftarrow \text{STFT}(y)$;
  $L \leftarrow 20 \log_{10}(|D|)$;
  $\theta \leftarrow \text{Otsu}(L) + 6$;
  $\mathbf{f}_i \leftarrow \text{Flatten}(\text{Resize}(L > \theta,\ 32 \times 128))$;
`// Phase 2: LSH candidate retrieval`
$\text{Sig} \leftarrow \text{MinHash}([\mathbf{f}_1, \ldots, \mathbf{f}_n]^\top,\ b \cdot r)$;
$\text{Cand} \leftarrow \text{BandCollisions}(\text{Sig},\ b,\ r)$;
$R \leftarrow \emptyset$; `// Phase 3: verification`
foreach $(i, j) \in \mathit{Cand}$ **do**
  if $\frac{|\mathbf{f}_i \cap \mathbf{f}_j|}{|\mathbf{f}_i \cup \mathbf{f}_j|} \geq \tau$ **then**
    $R \leftarrow R \cup \{\text{RandDrop}(i, j)\}$;
return $\mathcal{A} \setminus R$

Table 3: Summary of evaluation datasets.

Dataset	Modality	Size	Files	Ground Truth
Mixed-Challenge	Mixed	–	12K	Classification labels
ImageNet-Expanded	Image	450GB	~3.8M	66% duplicates
ESC-Expanded	Audio	20GB	~66K	80% duplicates
Amazon-Expanded	Text	500GB	–	80% duplicates
Mixed-Test	Mixed	2.5GB	17.6K	Per-modality labels

4 EXPERIMENTAL SETUP

We evaluate MMdedup through comprehensive experiments designed to answer four research questions: (RQ1) How accurately can the Sorter classify mixed-modality files? (RQ2) How does each modality-specific deduplication module compare against existing methods? (RQ3) How do similarity thresholds affect deduplication rates and downstream task performance? (RQ4) What is the contribution of each component to the overall system?

4.1 Datasets

We construct five datasets to evaluate different aspects of MMdedup, as summarized in Table 3.

To stress-test file classification, we create the Mixed-Challenge dataset of 12,000 files with deliberately adversarial characteristics, including misleading extensions (e.g., JPEG images named with a `.txt` suffix), JSON-wrapped modality references containing `image_url` or `audio_path` fields, and ambiguous binary content. For ablation studies, we construct a separate mixed-modality set, Mixed-Test (17,579 files, 2.5 GB), containing 7,800 images, 4,365 audio files, 3,340 text documents, and 2,074 unknown-type files, each subset seeded with controlled duplicates.

The three deduplication benchmarks are built by augmenting established datasets with known ground-truth duplicates. We expand ImageNet-1K [46] from 150 GB to 450 GB (ImageNet-Expanded) by generating three variants per source image: an exact binary copy, a transcoded PNG with compression artifacts, and a geometrically transformed JPEG with random rotation ($\pm 15^\circ$) and scaling (0.8–1.2×), yielding approximately 66% duplicates of varying difficulty. For audio and text, we adopt a shared expansion protocol of 80% exact copies and 20% modality-appropriate near-duplicates: ESC-50 [38] is extended from 600 MB to 20 GB (ESC-Expanded) with white-noise injection (SNR 20–40 dB) and temporal cropping (5–15% of duration), while the Amazon Product Reviews dataset [37] is scaled from 1.6 GB to 500 GB (Amazon-Expanded) with appended random byte sequences (8–32 bytes) that alter file hashes while preserving text content.

4.2 Baselines

We compare MMdedup against representative methods for each task.

File Classification. We evaluate three baselines: (1) Naive Extension, which classifies files solely by their filename extension; (2) libmagic [12], the widely-deployed library underlying the Linux `file` command, which identifies file types through magic number signatures; and (3) the file command itself, representing the standard Unix utility for file type detection.

Image Deduplication. We compare against four methods spanning exact matching to semantic similarity: (1) MD5 Hash, which detects only bit-identical duplicates; (2) pHash [56], a perceptual hashing method based on discrete cosine transform coefficients; (3) SimCLR [8], which uses contrastive learning embeddings for similarity computation; and (4) SemDeDup [1], the state-of-the-art semantic deduplication method that clusters images by CLIP embeddings and removes samples closest to cluster centroids.

Audio Deduplication. We compare against: (1) MD5 Hash for exact matching; and (2) MFCC+Cosine, a traditional approach that extracts Mel-frequency cepstral coefficients and computes pairwise cosine similarity.

Text Deduplication. We compare against three methods: (1) MD5 Hash for exact matching; (2) MinHash+LSH [2], the industry-standard method used in large-scale text deduplication pipelines including C4 [42] and The Pile [20]; and (3) SimHash [7], Google's locality-sensitive hashing scheme for near-duplicate web page detection.

4.3 Evaluation Metrics

We evaluate MMdedup across four dimensions: classification quality, deduplication quality, computational efficiency, and downstream task impact.

Classification Quality. To assess the Sorter's ability to correctly route files to modality-specific pipelines, we report two complementary metrics. Accuracy measures the fraction of files assigned to the correct modality category:

$$\text{Accuracy} = \frac{|\{f \in \mathcal{F} : \hat{y}_f = y_f\}|}{|\mathcal{F}|} \tag{3}$$

where $\mathcal{F}$ denotes the set of input files, y_f is the ground-truth modality label, and $\hat{y}_f$ is the predicted label. We also report *Macro F1-score*, which computes the harmonic mean of precision and recall for each class $c \in C$ and averages across classes:

$$\text{Macro F1} = \frac{1}{|C|} \sum_{c \in C} \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \tag{4}$$

where $P_c = \text{TP}_c/(\text{TP}_c + \text{FP}_c)$ and $R_c = \text{TP}_c/(\text{TP}_c + \text{FN}_c)$ are the per-class precision and recall, respectively. Macro F1 provides a balanced assessment across modalities, preventing dominant classes from masking poor performance on minority classes.

Deduplication Quality. We measure the effectiveness of duplicate removal using three metrics. Deduplication Rate quantifies the fraction of data identified and removed as duplicates:

$$\text{Dedup Rate} = \frac{|\mathcal{D}_{\text{removed}}|}{|\mathcal{D}|} \times 100\% \tag{5}$$

where $\mathcal{D}$ is the original dataset and $\mathcal{D}_{\text{removed}}$ is the set of files marked as duplicates. To assess detection correctness against ground-truth duplicate sets $\mathcal{G}$, we compute Precision and Recall:

$$\text{Precision} = \frac{|\mathcal{D}_{\text{removed}} \cap \mathcal{G}|}{|\mathcal{D}_{\text{removed}}|}, \quad \text{Recall} = \frac{|\mathcal{D}_{\text{removed}} \cap \mathcal{G}|}{|\mathcal{G}|} \tag{6}$$

High precision ensures that unique, informative samples are preserved rather than erroneously discarded, while high recall ensures comprehensive removal of redundant content. For datasets without explicit ground-truth labels, we construct $\mathcal{G}$ through our controlled data expansion process, which generates known duplicates with exact and near-duplicate variants.

Computational Efficiency. We report three efficiency metrics to characterize computational costs. Throughput measures processing speed in files per second (for classification) or domain-specific units (images/second for image deduplication). Processing Time reports end-to-end wall-clock time for each pipeline stage. GPU Memory records peak memory consumption during embedding extraction and similarity computation, which determines the feasibility of deployment on resource-constrained hardware.

Downstream Task Impact. To validate that deduplication preserves or improves data utility for model training, we train standard classification models on deduplicated datasets and report test accuracy on held-out evaluation sets.

$$\text{Test Acc} = \frac{|\{x \in \mathcal{T} : \hat{y}_x = y_x\}|}{|\mathcal{T}|} \tag{7}$$

where $\mathcal{T}$ denotes the test set. This metric directly measures whether deduplication harms model generalization (accuracy drop) or improves it (accuracy gain through noise removal). A key insight from our experiments is that aggressive deduplication can actually improve downstream accuracy by eliminating near-duplicates that cause data leakage between training and test splits.

4.4 Implementation Details

Deduplication experiments are conducted on a server equipped with an NVIDIA RTX 3090 GPU (24GB VRAM), Intel Xeon Gold 6226R CPU, and 64GB RAM. The Sorter classification benchmark runs on a separate workstation with an Intel Core i5-13490F CPU (10 cores, 20 threads) and 32GB RAM, running Windows 11 Pro with Python 3.11.

We implement MMdedup in Python 3.10 using PyTorch 2.0 for GPU-accelerated computation. Image embeddings are extracted using OpenCLIP's ViT-B/16 model pretrained on LAION-2B [47],

Table 4: File classification performance comparison.

Method	Accuracy (%)	Macro F1	Throughput (K files/s)	Memory (MB)
Naive Extension	62.10	0.61	14.97	2.36
Libmagic	50.99	0.49	1.35	1.82
File Command	50.99	0.49	1.32	1.47
Our Sorter	**100.00**	**1.00**	4.25	4.03

audio processing uses Librosa 0.10 for spectrogram computation, text hashing employs the `datasketch` library for MinHash signatures, and all similarity searches leverage Faiss [26] for efficient nearest neighbor retrieval.

Unless otherwise specified, we use the following default hyperparameters: $k = 2000$ clusters and similarity threshold $\epsilon = 0.1$ (cosine similarity ≥ 0.9) for image deduplication; $b = 20$ bands, $r = 10$ rows, and Jaccard threshold $\tau = 0.7$ for audio deduplication; and 128 hash permutations with Jaccard threshold $\tau = 0.8$ for text deduplication. These values were selected through preliminary experiments on validation splits.

For downstream evaluation, we train ResNet-18 [23] on ImageNet-Expanded with full fine-tuning, batch size 512, learning rate 0.0001 (Adam), for 2 epochs. On ESC-Expanded, we fine-tune ImageNet-pretrained ResNet-18 on 128-bin mel-spectrograms resized to 224×224, with batch size 32, learning rate 0.001 (Adam), for 10 epochs using an 80/20 train/test split. For Amazon-Expanded, we employ online learning with HashingVectorizer (2^{20} features) and SGDClassifier trained over 3 epochs; we reserve 20% of original data as a fixed validation set via deterministic content-based hashing, ensuring fair comparison across methods. All experiments use random seed 42 for reproducibility.

5 EVALUATION RESULTS

We present comprehensive experimental results addressing the four research questions outlined in Section 4. We first evaluate the Sorter's classification accuracy (RQ1), then compare each modality-specific deduplication module against existing methods (RQ2), analyze threshold sensitivity and downstream task impact (RQ3), and finally conduct ablation studies to quantify each component's contribution (RQ4).

5.1 File Classification Performance

In Table 4, we present the classification performance on the Mixed-Challenge dataset, which contains 12,000 files with deliberately adversarial characteristics, including extension mismatches and JSON-wrapped modality references.

The Naive Extension baseline achieves only 62.10% accuracy, failing entirely on files with misleading extensions. More surprisingly, both libmagic and the standard `file` command achieve even lower accuracy (50.99%), despite their reliance on binary magic numbers rather than extensions. This degradation occurs because these tools correctly identify JSON-wrapped files as "text/plain" based on their file headers, but fail to recognize that these files semantically represent image or audio references through fields such as `image_url` or `audio_path`.

Table 5: Image deduplication performance comparison.

Method	Dedup (%)	Prec (%)	Rec (%)	TP (img/s)	GPU (GB)	Acc (%)	Time (h)
No Dedup.	0.00	–	–	–	–	79.94	–
MD5	0.30	100.00	0.00	2640.2	0	80.10	6.76
pHash [56]	60.39	100.00	99.97	356.2	0	72.26	6.33
SimCLR [8]	69.17	18.26	99.68	45.0	21.3	67.80	11.8
SemDeDup [1]	69.21	93.70	96.20	27.9	4.0	67.86	12.0
Ours (CLIP)	**67.25**	85.24	69.25	**101.2**	4.6	**69.58**	10.5

Our Sorter achieves 100% accuracy and perfect Macro F1 score of 1.0 by employing a multi-layered hybrid heuristic strategy. The key insight is combining traditional magic number detection with JSON payload analysis that examines semantic field patterns. While throughput (4.25 K/s) is lower than the trivial extension-based approach, it is 3.1× faster than libmagic while providing substantially higher accuracy. The modest memory overhead (4.03 MB vs. 1.47–2.36 MB) is negligible for practical deployments.

5.2 Deduplication Effectiveness

We evaluate our modality-specific deduplication modules against established baselines, measuring both deduplication quality (precision, recall) and efficiency (throughput, time).

5.2.1 Image Deduplication. Table 5 compares image deduplication methods on ImageNet-Expanded (450GB), which contains approximately 66% duplicates including exact copies, transcoded variants, and geometric transformations.

MD5 hashing detects only exact binary duplicates, achieving a mere 0.30% deduplication rate and completely missing the near-duplicates that constitute the majority of redundancy. Perceptual hashing (pHash) achieves high precision (100%) and recall (99.97%) but aggressively removes 60.39% of data, resulting in significant downstream accuracy degradation (72.26% vs. 79.94% baseline). This indicates that pHash incorrectly marks semantically distinct images as duplicates based on superficial visual similarity.

Among semantic methods, SimCLR achieves high recall (99.68%) but extremely low precision (18.26%), indicating that its embedding space does not adequately separate semantically distinct images. SemDeDup [1] provides better precision (93.70%) but suffers from low throughput (27.9 imgs/s) due to its computationally intensive clustering approach.

Our CLIP-based method achieves the highest downstream test accuracy among semantic deduplication methods (69.58%), outperforming both SimCLR (67.80%) and SemDeDup (67.86%), while maintaining 3.6× higher throughput than SemDeDup. Although our precision (85.24%) and recall (69.25%) appear lower than some baselines, the downstream accuracy metric demonstrates that our method optimally balances duplicate removal with information preservation. The key advantage lies in our hierarchical clustering strategy (Algorithm 2), which reduces the $O(n^2)$ pairwise comparison to $O(n^{3/2})$ complexity while preserving semantic coherence within clusters.

5.2.2 Audio Deduplication. Table 6 presents results on ESC-Expanded (20GB), containing 80% exact duplicates and 20% acoustic perturbations including white noise injection and temporal cropping.

Table 6: Audio deduplication performance comparison.

Method	Dedup. (%)	Prec. (%)	Recall (%)	Time (s)	Test Acc. (%)
No Dedup.	0.00	–	–	–	99.97†
MD5	69.64	100.00	87.44	228	90.37
MFCC+Cosine [34]	98.68	79.65	98.69	47	50.94
Ours (LSH)	**77.48**	90.77	88.32	86	**92.01**

† The 99.97% test accuracy is artificially inflated due to duplicate leakage across the train/test split.

Table 7: Text deduplication performance comparison.

Method	Dedup. (%)	Prec. (%)	Recall (%)	Time (h)	Test Acc. (%)
No Dedup.	0.00	–	–	–	83.21
MD5	90.01	100.00	78.60	12.5	83.14
MinHash+LSH [2]	90.03	99.95	98.75	70	83.14
SimHash [7]	7.17	50.00	0.10	120	83.22
Ours (Multi)	90.03	99.95	**99.05**	**60**	83.14

The No Dedup baseline shows 99.97% test accuracy, but this metric is misleading: duplicates spanning the train/test split cause severe data leakage, allowing the model to "memorize" test samples rather than learn generalizable features. This observation underscores the critical importance of deduplication for reliable model evaluation.

MD5 hashing removes 69.64% of data (the exact duplicates) with perfect precision, yielding 90.37% test accuracy after eliminating the leakage effect. However, MD5 misses all acoustically perturbed near-duplicates (recall limited to 87.44% of ground-truth duplicates).

MFCC with cosine similarity achieves high recall (98.69%) but over-aggressively removes 98.68% of the data, leading to a catastrophic drop in accuracy to 50.94% on the 50-class ESC dataset, barely above random chance, which indicates that traditional audio features lack the robustness to distinguish semantically different sounds that share similar spectral characteristics.

Our spectral fingerprinting approach with LSH achieves the highest downstream accuracy (92.01%), surpassing even the MD5 baseline by 1.64 percentage points. This improvement occurs because our method successfully removes both exact duplicates and acoustically similar near-duplicates that would otherwise cause subtle data omissions, while preserving semantically distinct samples. The 77.48% deduplication rate represents an optimal balance, validated by the superior generalization performance.

5.2.3 Text Deduplication. Table 7 presents results on Amazon-Expanded (500GB), containing 80% exact duplicates and 20% near-duplicates with appended byte noise.

MD5 achieves a 90.01% deduplication rate by capturing exact duplicates, but it still misses 21.40% of the ground-truth duplicates, namely near-duplicates with byte noise, resulting in a recall of 78.60%. SimHash performs poorly on this dataset, reaching only 7.17% deduplication with 50% precision, which is effectively random. We attribute this failure to SimHash's sensitivity to the particular byte-noise perturbations in our dataset, which change the hash values even when the semantic content is preserved.

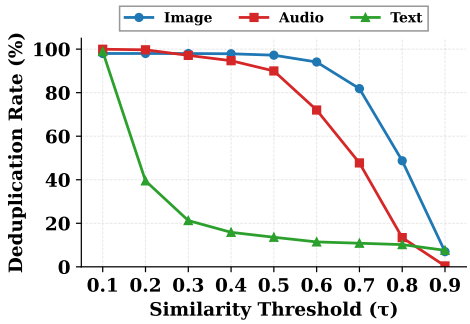


Figure 3: Deduplication rate as a function of similarity threshold.

MinHash+LSH [2] serves as the industry-standard baseline, achieving 99.95% precision and 98.75% recall. Our multi-granularity N-gram method matches this quality (99.95% precision, 99.05% recall) while reducing processing time by 14% (60h vs. 70h). The efficiency gain stems from our sample-based comparison strategy (Algorithm 1), which limits pairwise comparisons to recent samples rather than the full corpus, reducing complexity from $O(n^2)$ to $O(n \cdot S)$ where $S \ll n$.

All effective deduplication methods (MD5, MinHash, Ours) yield identical downstream accuracy (83.14%), marginally below the No Dedup baseline (83.21%). This minimal impact suggests that the Amazon Reviews dataset, even with 90% redundancy, contains sufficient unique information for model training. The key benefit of deduplication in this case is the 10× reduction in storage and training time rather than accuracy improvement.

5.3 Threshold Sensitivity Analysis

We illustrate how deduplication rates vary with the similarity threshold τ across all three modalities in Figure 3. The three modalities exhibit distinct sensitivity patterns.

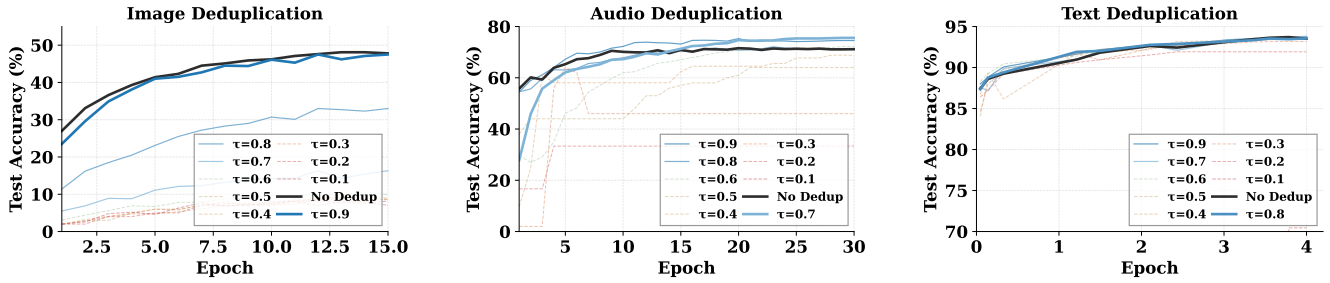
Image deduplication maintains near-constant rates (>97%) for $\tau \leq 0.6$, then drops sharply: from 81.9% at $\tau = 0.7$ to 48.7% at $\tau = 0.8$ and merely 7.0% at $\tau = 0.9$. This steep transition reflects the semantic clustering structure of ImageNet, where images within the same class share high similarity while cross-class pairs are well-separated.

Audio deduplication shows similar behavior but with the transition occurring at lower thresholds: rates exceed 90% for $\tau \leq 0.5$, then decline through 47.7% at $\tau = 0.7$ to 0.4% at $\tau = 0.9$. The earlier transition suggests that audio spectral fingerprints exhibit greater intra-class variance than image embeddings.

Text deduplication displays markedly different characteristics, with rates ranging only from 39.5% at $\tau = 0.2$ to 7.6% at $\tau = 0.9$. This compressed range indicates that the N-gram similarity distribution is more uniform, lacking the bimodal structure observed in image and audio modalities. Consequently, text deduplication is relatively insensitive to threshold selection.

5.4 Impact on Downstream Tasks

As shown in Figure 4, we present downstream model performance across different deduplication thresholds, enabling principled threshold selection by balancing data reduction against task performance.



(a) Image modality **(b) Audio modality** **(c) Text modality**

Figure 4: Downstream task performance as a function of training epochs for different similarity thresholds.

Table 8: Ablation study on Mixed-Test.

Configuration	Image (%)	Audio (%)	Text (%)	Overall (%)	Time (m)	Save (MB)
Full MMdedup	48.32	39.77	47.72	46.13	17.5	950
w/o Sorter (Image only)	47.67	0.00	0.00	21.15	1.9	200
w/o Sorter (Audio only)	0.00	39.82	0.00	9.89	17.1	731
w/o Sorter (Text only)	0.00	0.00	47.72	9.07	0.3	0.5
w/o Sorter (Ground-truth)	48.32	39.89	47.72	46.16	17.6	952
w/o Near-Dedup	0.00	37.14	47.72	25.52	2.1	529
w/o Image Dedup	–	39.40	47.72	44.44	15.9	724
w/o Audio Dedup	50.19	–	47.72	49.05	1.9	223
w/o Text Dedup	47.67	39.79	–	44.84	17.5	931

For image data (Figure 4a), aggressive thresholds ($\tau \leq 0.8$) cause severe accuracy degradation, with $\tau = 0.5$ yielding only 9% final accuracy compared to 47.8% for the undeduped baseline. The conservative threshold $\tau = 0.9$ preserves accuracy (47.5%) while still removing approximately 7% of duplicates. We therefore recommend $\tau^* = 0.9$ for image deduplication, which achieves the optimal balance between storage reduction and performance preservation.

Audio deduplication (Figure 4b) reveals a striking result: $\tau = 0.7$ consistently *outperforms* the undeduped baseline, achieving 75.6% accuracy versus 71.2%. This counterintuitive improvement occurs because our method effectively removes "harmful" near-duplicates, namely acoustically similar samples that cause data leakage and prevent the model from learning robust features. More aggressive thresholds ($\tau \leq 0.5$) remove too much data, degrading performance. We recommend $\tau^* = 0.7$, which achieves 47.7% data reduction while improving downstream accuracy.

Text deduplication (Figure 4c) shows remarkable insensitivity to threshold selection, with all settings achieving approximately 93–94% final accuracy, nearly identical to the baseline. This stability allows practitioners to choose thresholds based on storage constraints rather than accuracy concerns. We recommend $\tau^* = 0.8$ as a conservative default that achieves approximately 10% deduplication while matching baseline performance.

5.5 Ablation Study

Table 8 quantifies the contribution of each MMdedup component on the Mixed-Test dataset (2.5GB, 17.6K files). We systematically disable individual components to isolate their effects on deduplication effectiveness and efficiency.

Impact of the Sorter. We evaluate the Sorter's contribution through two complementary ablations. First, replacing predicted classifications with ground-truth labels (w/o Sorter, Ground-truth) yields nearly identical results to the full pipeline (46.16% vs. 46.13% overall deduplication), confirming that our Sorter introduces negligible classification errors on this dataset. Second, we simulate scenarios where the Sorter is absent and only a single modality deduplication algorithm can be processed. When restricted to image-only deduplication processing, the system achieves only 21.15% overall deduplication despite removing 47.67% of images, because it is ineffective for other two modalities, audio and text files remain entirely unprocessed. Similarly, audio-only and text-only configurations achieve merely 9.89% and 9.07% overall deduplication, respectively. These results demonstrate that the Sorter is essential for multimodal scenarios: without accurate classification, a deduplication pipeline can only target one modality, forfeiting the majority of potential storage savings.

Impact of Near-Duplicate Detection. Disabling semantic similarity detection and relying solely on hash-based exact matching (w/o Near-Dedup) reduces overall deduplication from 46.13% to 25.52%, a 20.61 percentage-point drop. The impact is most pronounced for images, where deduplication falls from 48.32% to 0%. This occurs because our ImageNet-Expanded dataset contains no exact binary duplicates; all redundancy exists as near-duplicates due to transcoded formats and geometric transformations. In contrast, text deduplication remains unchanged at 47.72%, since the Amazon-Expanded dataset contains substantial exact duplicates detectable by MD5 hashing. Audio deduplication decreases modestly from 39.77% to 37.14%, reflecting its mixed composition of exact and perturbed duplicates. These findings validate our core design principle: effective multimodal deduplication requires semantic similarity detection beyond hash-based exact matching, particularly for image data where near-duplicates dominate.

Impact of Modality-Specific Modules. Removing individual deduplication modules reveals their complementary contributions. Disabling image deduplication (w/o Image Dedup) reduces storage savings from 950MB to 724MB, while decreasing processing time from 17.5m to 15.9m, indicating that image processing contributes moderate computational overhead relative to its storage benefit. Disabling audio deduplication (w/o Audio Dedup) dramatically reduces both time (17.5m to 1.9m) and storage savings (950MB to 223MB), reflecting audio's dominance in file size within this dataset. Disabling

text deduplication (w/o Text Dedup) has minimal time impact (17.5m to 17.5m) but reduces storage savings from 950MB to 931MB, consistent with text files being numerous but individually small. Notably, removing one modality's deduplication slightly increases deduplication rates for other modalities (e.g., image deduplication rises from 48.32% to 50.19% when audio is disabled), likely due to reduced memory pressure enabling more thorough processing.

The ablation results collectively validate our modular architecture. Each component, including the Sorter, near-duplicate detection, and modality-specific modules, contributes meaningfully to overall effectiveness. The full MMdedup pipeline delivers synergistic benefits with 46.13% deduplication and 950MB savings that no single-modality approach can match, demonstrating the value of an integrated, end-to-end multimodal data-cleaning approach.

6 DISCUSSION

We discuss the efficiency characteristics and scalability potential of MMdedup, followed by an assessment of current limitations and directions for future work.

6.1 Efficiency and Scalability

A key design goal of MMdedup is to achieve practical efficiency for real-world data engineering pipelines. The classification stage reaches 4.25 K files per second on a single CPU core, processing the 12K Mixed-Challenge dataset in under 3 seconds. Memory consumption remains bounded at approximately 4 MB regardless of dataset size because files are processed in a streaming fashion. The lightweight heuristic design, which combines magic number detection with selective JSON parsing, avoids the computational overhead of deep learning classifiers while maintaining perfect accuracy on adversarial inputs.

The three modality-specific deduplication modules exhibit different computational profiles. Image deduplication achieves 101.2 images/second on a single RTX 3090 GPU, with the hierarchical clustering approach reducing complexity from $O(n^2)$ to $O(n^{3/2})$ and enabling processing of our 450GB dataset in 10.5 hours. Audio deduplication processes 20GB in 86 seconds (767 files/second), leveraging CPU-bound spectral fingerprinting with $O(n \log n)$ LSH indexing. Text deduplication handles the largest volume (500GB in 60 hours), achieving 14% speedup over MinHash+LSH through sample-based comparison that limits complexity to $O(n \cdot S)$.

The modular architecture offers natural parallelization at multiple levels. After classification, the three pipelines operate on disjoint subsets and can execute concurrently. Each module further supports intra-modality parallelism: image embedding scales across GPUs, audio fingerprinting distributes across CPU cores, and text processing supports chunk-based parallelism. For distributed deployment, hash-based partitioning on cluster or LSH bucket assignments maintains locality while enabling horizontal scaling. Based on our experiments, we estimate that a modest 4-node GPU cluster could process petabyte-scale datasets within practical time budgets, making MMdedup viable for industrial MLLM training data preparation.

6.2 Limitations and Future Work

Our current implementation has several limitations. The text and audio modules prioritize efficiency over deep semantic understanding. For example, N-gram similarity captures syntactic patterns but misses paraphrases, while spectral fingerprinting identifies acoustic duplicates but cannot detect semantically equivalent recordings with different acoustic signatures. The pipeline treats modalities independently and may miss cross-modal duplicates when identical content appears in different formats. In addition, optimal similarity thresholds vary across modalities and datasets, so they currently require empirical validation rather than automatic selection. Finally, MMdedup operates in batch mode and does not support incremental deduplication for streaming data ingestion.

Future work could address these limitations through several directions: hybrid approaches applying lightweight filtering followed by selective deep semantic analysis; adaptive threshold selection using AutoML techniques; multimodal embeddings for cross-modal duplicate detection; incremental indexing with Bloom filters for streaming scenarios; and quality-aware selection that retains the highest-quality duplicate rather than arbitrary choice. The modular design of MMdedup facilitates such incremental improvements, and individual components can be upgraded without restructuring the overall pipeline.

7 CONCLUSIONS

We presented MMdedup, an end-to-end framework for multimodal data deduplication designed to clean the "digital swamp" of raw, heterogeneous web data for MLLM training. Our approach addresses a critical gap in existing data cleaning pipelines: the absence of an integrated solution that first classifies mixed-modality inputs and then applies modality-aware deduplication. The framework consists of two stages: 1) The Sorter stage employs a lightweight heuristic classifier combining magic number detection with JSON semantic parsing, achieving 100% accuracy on adversarial inputs while maintaining high throughput (4.25 K files/second); 2) The deduplication stage applies tailored algorithms for each modality: CLIP-based semantic clustering for images, spectral fingerprinting with LSH for audio, and multi-granularity N-gram matching for text. Each algorithm is designed to balance deduplication quality against computational efficiency at scale.

Our experimental evaluation on large-scale datasets demonstrates that MMdedup effectively removes both exact and near-duplicate content while preserving data utility for downstream MLLM training tasks. Key findings include: 1) the Sorter outperforms standard tools (libmagic, file command) by correctly handling JSON-wrapped modality references; 2) our image deduplication achieves 3.6× higher throughput than SemDeDup while improving downstream accuracy; 3) audio deduplication can achieve the highest downstream accuracy 92.01% with an optimal balance between deduplication rate and performance; and 4) text deduplication matches MinHash quality with 14% faster processing. Ablation studies confirm that near-duplicate detection contributes over 20 percentage points of additional deduplication beyond exact matching, validating our core design principle that semantic similarity detection is essential for effective multimodal data cleaning.

As multimodal AI systems continue to scale, the demand for high-quality training data will only intensify. MMdedup provides a practical, validated solution for transforming chaotic web harvests into clean, deduplicated corpora, contributing to the broader goal of efficient and responsible AI development.

REFERENCES

[1] Amro Abbas, Kushal Tirumala, Dániel Simig, Surya Ganguli, and Ari S Morcos. 2023. Semdedup: Data-efficient learning at web-scale through semantic deduplication. *arXiv preprint arXiv:2303.09540* (2023).

[2] Andrei Z Broder. 1997. On the resemblance and containment of documents. In *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No. 97TB100171)*. IEEE, 21–29.

[3] Andrei Z Broder, Moses Charikar, Alan M Frieze, and Michael Mitzenmacher. 1998. Min-wise independent permutations. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 327–336.

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[5] Andrés Eduardo Castro-Ospina, Miguel Angel Solarte-Sanchez, Laura Stella Vega-Escobar, Claudia Isaza, and Juan David Martínez-Vargas. 2024. Graph-Based Audio Classification Using Pre-Trained Models and Graph Neural Networks. *Sensors* 24, 7 (2024). https://doi.org/10.3390/s24072106

[6] Sungkyun Chang, Donmoon Lee, Jeongsoo Park, Hyungui Lim, Kyogu Lee, Karam Ko, and Yoonchang Han. 2021. Neural audio fingerprint for high-specific audio retrieval based on contrastive learning. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 3025–3029.

[7] Moses S Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *Proceedings of the thiry-fourth annual ACM symposium on Theory of computing*. 380–388.

[8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*. PmLR, 1597–1607.

[9] Yang Chen, Hairu Wen, Yan Li, and Yanfei Ma. 2025. Syntheclean: Enhancing large-scale multimodal models via adaptive data synthesis and cleaning. In *2025 5th International Conference on Artificial Intelligence and Industrial Technology Applications (AIITA)*. IEEE, 1769–1772.

[10] Xu Chu, Ihab F Ilyas, Sanjay Krishnan, and Jiannan Wang. 2016. Data cleaning: Overview and emerging challenges. In *Proceedings of the 2016 international conference on management of data*. 2201–2206.

[11] Michele Dallachiesa, Amr Ebaid, Ahmed Eldawy, Ahmed Elmagarmid, Ihab F Ilyas, Mourad Ouzzani, and Nan Tang. 2013. NADEEF: a commodity data cleaning system. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. 541–552.

[12] Ian F Darwin. 2008. libmagic—File type identification library. https://github.com/file/file.

[13] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry*. ACM, 253–262.

[14] Inderjit S. Dhillon and Dharmendra S. Modha. 2001. Concept Decompositions for Large Sparse Text Data Using Clustering. *Machine Learning* 42, 1-2 (2001), 143–175. https://doi.org/10.1023/A:1007612920971

[15] Xiaoou Ding, Zekai Qian, Hongzhi Wang, Siying Chen, Yafeng Tang, Hongbin Su, Huan Hu, and Chen Wang. 2025. UniClean: A Scalable Data Cleaning Solution for Mixed Errors Based on Unified Cleaners and Optimized Cleaning Workflow. *Proceedings of the VLDB Endowment* 18, 11 (July 2025), 4117–4130. https://doi.org/10.14778/3749646.3749681

[16] Matthijs Douze, Hervé Jégou, Jeff Johnson, Giorgos Tolias, and Ivan Turrisi. 2017. Faiss: A Library for Efficient Similarity Search and Clustering of Dense Vectors. *arXiv preprint arXiv:1702.08734* (February 2017). http://arxiv.org/abs/1702.08734

[17] Yanick Fratantonio, Luca Invernizzi, Loua Farah, Kurt Thomas, Marina Zhang, Ange Albertini, Francois Galilee, Giancarlo Metitieri, Julien Cretin, Alex Petit-Bianco, et al. 2024. Magika: AI-Powered Content-Type Detection. *arXiv preprint arXiv:2409.13768* (2024).

[18] Yinjin Fu, Jun Su, Jiahao Ning, Jian Wu, Yutong Lu, and Nong Xiao. 2025. Distributed Data Deduplication for Big Data: A Survey. *Comput. Surveys* 58, 3, Article 66 (Sept. 2025), 38 pages. https://doi.org/10.1145/3735508

[19] Samir Yitzhak Gadre, Gabriel Ilharco, Alex Fang, Jonathan Hayase, Georgios Smyrnis, Thao Nguyen, Ryan Marten, Mitchell Wortsman, Dhruba Ghosh, Jieyu Zhang, et al. 2023. Datacomp: In search of the next generation of multimodal datasets. *Advances in Neural Information Processing Systems* 36 (2023), 27092–27112.

[20] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. 2020. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027* (2020).

[21] Siddharth Gopal, Yiming Yang, Konstantin Salomatin, and Jaime Carbonell. 2011. Statistical learning for file-type identification. In *2011 10th international conference on machine learning and applications and workshops*, Vol. 1. IEEE, 68–73.

[22] Jindong Han, Hao Wang, Hui Xiong, and Hao Liu. 2025. Scalable Pre-Training of Compact Urban Spatio-Temporal Predictive Models on Large-Scale Multi-Domain Data. *Proc. VLDB Endow.* 18, 7 (March 2025), 2149–2158. https://doi.org/10.14778/3734839.3734851

[23] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

[24] Shawn Hershey, Sourish Chaudhuri, Daniel P. W. Ellis, Jort F. Gemmeke, Aren Jansen, R. Channing Moore, Manoj Plakal, Devin Platt, Rif A. Saurous, Bryan Seybold, Malcolm Slaney, Ron J. Weiss, and Kevin Wilson. 2017. CNN Architectures for Large-Scale Audio Classification. arXiv:1609.09430 [cs.SD] https://arxiv.org/abs/1609.09430

[25] Shubham Jain, Vincent Rimmer, Mika Juuti, Mauro Conti, and N. Asokan. 2023. Deep perceptual hashing algorithms with hidden dual purpose: when client-side scanning does facial recognition. *arXiv preprint arXiv:2306.11924* (June 2023). http://arxiv.org/abs/2306.11924

[26] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE transactions on big data* 7, 3 (2019), 535–547.

[27] Arham Khan, Robert Underwood, Carlo Siebenschuh, Yadu Babuji, Aswathy Ajith, Kyle Hippe, Ozan Gokdemir, Alexander Brace, Kyle Chard, and Ian Foster. 2024. Lshbloom: Memory-efficient, extreme-scale document deduplication. *arXiv preprint arXiv:2411.04257* (2024).

[28] Lukáš Lalinský. 2010. Chromaprint: Audio Fingerprint Library. https://acoustid.org/chromaprint.

[29] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2022. Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8424–8445.

[30] Jure Leskovec, Anand Rajaraman, and Jeffrey D. Ullman. 2014. *Mining of Massive Datasets*. Cambridge University Press.

[31] Wei-Jen Li, Ke Wang, Salvatore J Stolfo, and Benjamin Herzog. 2005. Fileprints: Identifying file types by n-gram analysis. In *Proceedings from the Sixth Annual IEEE SMC Information Assurance Workshop*. IEEE, 64–71.

[32] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A configuration-free error detection system. In *Proceedings of the 2019 International Conference on Management of Data*. 865–882.

[33] Mason McDaniel and Mohammad Hossain Heydari. 2003. Content based file type detection algorithms. In *36th Annual Hawaii International Conference on System Sciences, 2003. Proceedings of the*. IEEE, 10–pp.

[34] Brian McFee, Colin Raffel, Dawen Liang, Daniel PW Ellis, Matt McVicar, Eric Battenberg, Oriol Nieto, et al. 2015. librosa: Audio and music signal analysis in python. *SciPy* 2015, 18-24 (2015), 7.

[35] Athicha Muthitacharoen, Benjie Chen, and David Mazieres. 2001. A low-bandwidth network file system. In *Proceedings of the eighteenth ACM symposium on Operating systems principles*. 174–187.

[36] Zan Ahmad Naeem, Mohammad Shahmeer Ahmad, Mohamed Eltabakh, Mourad Ouzzani, and Nan Tang. 2024. RetClean: Retrieval-Based Data Cleaning Using LLMs and Data Lakes. *Proceedings of the VLDB Endowment* 17, 12 (Aug. 2024), 4421–4424. https://doi.org/10.14778/3685800.3685890

[37] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 188–197.

[38] Karol J Piczak. 2015. ESC: Dataset for environmental sound classification. In *Proceedings of the 23rd ACM international conference on Multimedia*. 1015–1018.

[39] Michael O Rabin. 1981. Fingerprinting by random polynomials. *Technical report* (1981).

[40] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PmLR, 8748–8763.

[41] Edward Raff, Jon Barker, Jared Sylvester, Robert Brandon, Bryan Catanzaro, and Charles Nicholas. 2017. Malware detection by eating a whole exe. *arXiv preprint arXiv:1710.09435* (2017).

[42] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.

[43] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv:1908.10084 [cs.CL] https://arxiv.org/abs/1908.10084

[44] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: holistic data repairs with probabilistic inference. *Proceedings of the VLDB Endowment* 10, 11 (Aug. 2017), 1190–1201. https://doi.org/10.14778/3137628.3137631

[45] Ronald Rivest. 1992. *The MD5 message-digest algorithm*. Technical Report.

[46] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. 2015. Imagenet large scale visual recognition challenge. *International journal of computer vision* 115, 3 (2015), 211–252.

[47] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, et al. 2022. Laion-5b: An open large-scale dataset for training next generation image-text models. *Advances in neural information processing systems* 35 (2022), 25278–25294.

[48] Shaoxu Song, Fei Gao, Aoqian Zhang, Jianmin Wang, and Philip S. Yu. 2021. Stream Data Cleaning under Speed and Acceleration Constraints. *ACM Transactions on Database Systems* 46, 3, Article 10 (Sept. 2021), 44 pages. https://doi.org/10.1145/3465740

[49] Benzhao Tang, Shiyu Yang, Zhitao Shen, Wenjie Zhang, Xuemin Lin, and Zhihong Tian. 2025. LogLite: Lightweight Plug-and-Play Streaming Log Compression. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3757–3770.

[50] Kushal Tirumala, Daniel Simig, Armen Aghajanyan, and Ari Morcos. 2023. D4: Improving llm pretraining via document de-duplication and diversification. *Advances in Neural Information Processing Systems* 36 (2023), 53983–53995.

[51] Avery Wang et al. 2003. An industrial strength audio search algorithm.. In *Ismir*, Vol. 2003. Washington, DC, 7–13.

[52] Zige Wang, Wanjun Zhong, Yufei Wang, Qi Zhu, Fei Mi, Baojun Wang, Lifeng Shang, Xin Jiang, and Qun Liu. 2024. Data Management For Training Large Language Models: A Survey. arXiv:2312.01700 [cs.CL] https://arxiv.org/abs/2312.01700

[53] Wen Xia, Hong Jiang, Dan Feng, Fred Douglis, Philip Shilane, Yu Hua, Min Fu, Yucheng Zhang, and Yukun Zhou. 2016. A Comprehensive Study of the Past, Present, and Future of Data Deduplication. *Proc. IEEE* 104, 9 (2016), 1681–1710. https://doi.org/10.1109/JPROC.2016.2571298

[54] Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. 2024. A survey on multimodal large language models. *National Science Review* 11, 12 (11 2024), nwae403. https://doi.org/10.1093/nsr/nwae403

[55] Xiao Yu, Zexian Zhang, Feifei Niu, Xing Hu, Xin Xia, and John Grundy. 2024. What Makes a High-Quality Training Dataset for Large Language Models: A Practitioners' Perspective. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) *(ASE '24)*. Association for Computing Machinery, New York, NY, USA, 656–668. https://doi.org/10.1145/3691620.3695061

[56] Christoph Zauner. 2010. Implementation and benchmarking of perceptual image hash functions. (2010).

[57] Duzhen Zhang, Yahan Yu, Jiahua Dong, Chenxing Li, Dan Su, Chenhui Chu, and Dong Yu. 2024. MM-LLMs: Recent Advances in MultiModal Large Language Models. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*. Association for Computational Linguistics, 12401–12430. https://doi.org/10.18653/V1/2024.FINDINGS-ACL.738

[58] Xiu Zhang, Bilei Zhu, Linwei Li, Wei Li, Xiaoqiang Li, Wei Wang, Peizhong Lu, and Wenqiang Zhang. 2015. SIFT-based local spectrogram image descriptor: a novel feature for robust music identification. *EURASIP Journal on Audio, Speech, and Music Processing* 2015, 1 (2015), 1–12. https://doi.org/10.1186/s13636-014-0036-8